

Clean Code

Robert C. Martin

Pertemuan 14

Dosen Pengampu:
Zulkifli Arsyad, S.Kom., M.T.
Joe Lian Min, M.Eng
Muhammad Riza Alifi, S.T., M.T.



1. Chapter 1 - Clean Code
2. Chapter 2 - Meaningful Names
3. Chapter 3 - Functions
4. Chapter 4 - Comments
5. Chapter 5 - Formatting
6. Chapter 6 - Objects and Data Structures
7. Chapter 7 - Error Handling
8. Chapter 8 - Boundaries
9. Chapter 9 - Unit Tests
10. Chapter 10 - Classes



Apa Itu Clean code?

- Menurut Robert C. Martin (*Uncle Bob*), *clean code* adalah kode yang mudah dibaca, mudah dipahami, dan mudah diubah oleh programmer lain. Kode yang bersih menunjukkan bahwa programmer peduli terhadap kualitas software, bukan hanya membuat program berjalan.



Apa Itu Clean code?

Clean code memiliki ciri:

- Mudah dipahami tanpa penjelasan panjang
- Struktur rapi dan konsisten
- Tidak berlebihan
- Memiliki tujuan yang jelas
- Minim bug dan mudah diuji



PROFESIONALISME SEORANG PROGRAMMER

Programmer profesional tidak hanya fokus menyelesaikan fitur, tetapi juga **menjaga kualitas kode** agar tetap dapat dipelihara dalam jangka panjang.

MENGAPA PROFESIONALISME PENTING?



Bertanggung jawab terhadap kualitas software



Tidak asal membuat kode cepat selesai



Memikirkan dampak jangka panjang



Menjaga maintainability sistem

SEORANG PROGRAMMER PROFESIONAL



Menolak membuat kode berantakan hanya demi deadline



Melakukan refactoring ketika diperlukan



Menulis kode yang bisa dipahami tim lain



Menjaga standar coding



PROGRAMMER

Jangan membuat kode buruk hanya karena ingin cepat.



Kode berantakan hari ini akan menjadi masalah besar di masa depan.



DOKTER

Dokter tidak boleh mengabaikan kebersihan hanya karena pasien ingin operasi lebih cepat.



Kebersihan adalah tanggung jawab profesional demi keselamatan pasien jangka panjang.



DAMPAK JANGKA PANJANG

KODE BURUK HARI INI



- Sulit dipahami
- Banyak bug
- Perubahan sulit
- Produktivitas turun
- Biaya maintenance tinggi

PERBAIKAN KONSISTEN



- ✓ Refactoring rutin
- ✓ Kode lebih bersih
- ✓ Mudah dipahami
- ✓ Mudah diuji
- ✓ Mudah dikembangkan

SISTEM SEHAT DI MASA DEPAN



- Produktivitas tinggi
- Mudah menambah fitur
- Sedikit bug
- Tim kolaboratif
- Biaya maintenance rendah



Programmer hebat bukan hanya membuat kode bekerja, tetapi membuat kode tetap mudah dipahami dan dipelihara bertahun-tahun kemudian.

– Uncle Bob



Chapter 2 - Meaningful Names



PRINSIP NAMING YANG BAIK

Nama yang jelas membuat kode mudah dipahami, dipelihara, dan dikerjakan oleh tim lain.

1 Gunakan nama yang menjelaskan tujuan



Nama harus menjelaskan apa yang dilakukan atau disimpan.

✗ Buruk

```
int d;  
int c;  
string s;
```

✓ Baik

```
int elapsedDays;  
int customerCount;  
string customerName;
```



Orang lain membaca kode lebih sering daripada menulis kode.

2 Hindari disinformasi dan singkatan ambigu



Jangan gunakan nama yang membingungkan atau singkatan yang tidak umum.

✗ Buruk

```
int cnt;  
int usrId;  
bool flag;
```

✓ Baik

```
int itemCount;  
int userId;  
bool isActive;
```



Singkatan bisa berbeda arti bagi setiap orang.

3 Gunakan nama yang mudah dicari dan diucapkan



Nama yang mudah dicari di codebase dan mudah diucapkan saat berdiskusi akan meningkatkan komunikasi tim.

✗ Buruk

```
int calcExpDt;  
List tmpData;  
string genUsrInfo;
```

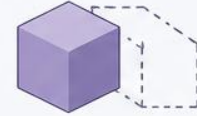
✓ Baik

```
int expirationDate;  
List temporaryData;  
string generateUserInfo;
```



Mudah dicari, mudah diucapkan, mudah dipahami.

4 Pilih satu kata untuk satu konsep



Jangan gabungkan beberapa konsep berbeda dalam satu nama.

✗ Buruk

```
int userDataList;  
string orderInfoDetail;  
bool isUpdateNew;
```

✓ Baik

```
User user;  
List<Order> orders;  
bool isUpdated;
```



Satu nama, satu tanggung jawab, satu konsep.

CONTOH KODE LENGKAP

✗ Kode dengan naming buruk

```
public void proc(int d, int c, string s, bool f) {  
    int i = 0;  
    while (i < c) {  
        string t = getDt(i);  
        if (f) {  
            save(t, s);  
        }  
        i++;  
    }  
}
```

// d = hari berlalu?
// c = jumlah?
// s = nama?
// f = kondisi?
// i = index?
// t = data?
// tidak jelas dan membingungkan



✓ Kode dengan naming baik

```
public void processOrder(int elapsedDays, int orderCount,  
    String customerName, boolean isActive) {  
    int currentIndex = 0;  
    while (currentIndex < orderCount) {  
        String orderData = getOrderData(currentIndex);  
        if (isActive) {  
            saveOrder(orderData, customerName);  
        }  
        currentIndex++;  
    }  
}
```

// jelas tujuannya
// mudah dipahami
// mudah dibaca
// mudah dipelihara



INGAT: Nama yang baik adalah dokumentasi terbaik. Kode yang baik berbicara sendiri.



PRINSIP NAMING YANG BAIK

Nama yang jelas membuat kode mudah dipahami, dipelihara, dan dikerjakan oleh tim lain.

1 Gunakan nama yang menjelaskan tujuan



Nama harus menjelaskan apa yang dilakukan atau disimpan.

✗ Buruk

```
int d;  
int c;  
string s;
```

✓ Baik

```
int elapsedDays;  
int customerCount;  
string customerName;
```



Orang lain membaca kode lebih sering daripada menulis kode.

2 Hindari disinformasi dan singkatan ambigu



Jangan gunakan nama yang membingungkan atau singkatan yang tidak umum.

✗ Buruk

```
int cnt;  
int usrId;  
bool flag;
```

✓ Baik

```
int itemCount;  
int userId;  
bool isActive;
```



Singkatan bisa berbeda arti bagi setiap orang.

3 Gunakan nama yang mudah dicari dan diucapkan



Nama yang mudah dicari di codebase dan mudah diucapkan saat berdiskusi akan meningkatkan komunikasi tim.

✗ Buruk

```
int calcExpDt;  
List tmpData;  
string genUsrInfo;
```

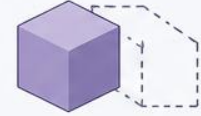
✓ Baik

```
int expirationDate;  
List temporaryData;  
string generateUserInfo;
```



Mudah dicari, mudah diucapkan, mudah dipahami.

4 Pilih satu kata untuk satu konsep



Jangan gabungkan beberapa konsep berbeda dalam satu nama.

✗ Buruk

```
int userDataList;  
string orderInfoDetail;  
bool isUpdateNew;
```

✓ Baik

```
User user;  
List<Order> orders;  
bool isUpdated;
```



Satu nama, satu tanggung jawab, satu konsep.

CONTOH KODE LENGKAP

✗ Kode dengan naming buruk

```
public void proc(int d, int c, string s, bool f) {  
    int i = 0;           // d = hari berlalu?  
    while (i < c) {       // c = jumlah?  
        string t = getDt(i); // s = nama?  
        if (f) {          // f = kondisi?  
            save(t, s);    // i = index?  
        }                 // t = data?  
        i++;              // tidak jelas dan membingungkan  
    }  
}
```



✓ Kode dengan naming baik

```
public void processOrder(int elapsedDays, int orderCount,  
    String customerName, boolean isActive) {  
    int currentIndex = 0;  
    while (currentIndex < orderCount) {  
        String orderData = getOrderData(currentIndex); // jelas tujuannya  
        if (isActive) { // mudah dipahami  
            saveOrder(orderData, customerName); // mudah dibaca  
        } // mudah dipelihara  
        currentIndex++;  
    }  
}
```



INGAT: Nama yang baik adalah dokumentasi terbaik. Kode yang baik berbicara sendiri.



3. Chapter 3 - Functions



Functions (*Clean Code*)

Prinsip penting

- Function harus kecil
- Fokus pada satu tugas
- Nama harus deskriptif
- Parameter minimal
- Hindari side effects

“Functions should do one thing. They should do it well.”

— Robert C. Martin



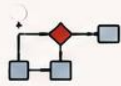
SLIDE 2 FUNCTION HARUS KECIL

Fungsi kecil lebih mudah dipahami, diuji, diperbaiki, dan digunakan ulang.

CIRI FUNCTION BURUK



Panjang



Banyak
nested if/loop



Banyak
tanggung jawab

❌ CONTOH PROGRAM BURUK

Function terlalu panjang dan memiliki banyak tanggung jawab

```
public class OrderServiceBad {  
    public void processOrder(Order order) {  
        // 1. Validasi input  
        if (order == null) {  
            throw new IllegalArgumentException("Order tidak boleh null");  
        }  
        if (order.getItems() == null || order.getItems().isEmpty()) {  
            throw new IllegalArgumentException("Item order tidak boleh kosong");  
        }  
        // 2. Hitung total  
        double total = 0;  
        for (Item item : order.getItems()) {  
            if (item.getQuantity() <= 0) {  
                throw new IllegalArgumentException("Quantity harus lebih dari 0");  
            }  
            total += item.getPrice() * item.getQuantity();  
        }  
        // 3. Terapkan diskon  
        if (order.getCustomer().isVip()) {  
            total = total * 0.9;  
        } else if (total >= 1000000) {  
            total = total * 0.95;  
        }  
        // 4. Simpan ke database  
        order.setTotal(total);  
        order.setStatus("PAID");  
        order.setOrderDate(LocalDate.now());  
        orderRepository.save(order);  
        // 5. Kirim email  
        emailService.sendInvoiceEmail(order.getCustomer().getEmail(), order);  
        // 6. Cetak invoice  
        pdfService.generateInvoice(order);  
    }  
}
```

MASALAH:

- ❌ Terlalu panjang (33+ baris)
- ❌ Melakukan banyak tanggung jawab (validasi, hitung, diskon, simpan, email, cetak)
- ❌ Sulit dipahami
- ❌ Sulit diuji (tidak bisa diuji per bagian)
- ❌ Sulit diperbaiki
- ❌ Sulit digunakan ulang



Satu perubahan kecil bisa berdampak ke banyak bagian lainnya.

KESIMPULAN BURUK



Function panjang membuat kode sulit dibaca, sulit diuji, sulit diubah, dan berisiko tinggi menimbulkan bug.

MANFAAT FUNCTION KECIL



Mudah
dipahami



Mudah
diuji



Mudah
diperbaiki



Mudah
digunakan ulang

✅ CONTOH PROGRAM BAIK

Function kecil, fokus pada satu tugas

```
public class OrderServiceGood {  
    public void processOrder(Order order) {  
        validateOrder(order);  
        double total = calculateTotal(order);  
        double discount = calculateDiscount(order, total);  
        double finalTotal = total - discount;  
        saveOrder(order, finalTotal);  
        sendInvoice(order);  
        generateInvoice(order);  
    }  
    // 1. Validasi  
    private void validateOrder(Order order) {  
        if (order == null) throw new IllegalArgumentException("Order tidak boleh null");  
        if (order.getItems() == null || order.getItems().isEmpty())  
            throw new IllegalArgumentException("Item order tidak boleh kosong");  
    }  
    // 2. Hitung total  
    private double calculateTotal(Order order) {  
        double total = 0;  
        for (Item item : order.getItems()) {  
            if (item.getQuantity() <= 0)  
                throw new IllegalArgumentException("Quantity harus lebih dari 0");  
            total += item.getPrice() * item.getQuantity();  
        }  
        return total;  
    }  
    // 3. Hitung diskon  
    private double calculateDiscount(Order order, double total) {  
        if (order.getCustomer().isVip()) return total * 0.10;  
        if (total >= 1000000) return total * 0.05;  
        return 0;  
    }  
    // 4. Simpan order  
    private void saveOrder(Order order, double finalTotal) {  
        order.setTotal(finalTotal);  
        order.setStatus("PAID");  
        order.setOrderDate(LocalDate.now());  
        orderRepository.save(order);  
    }  
    // 5. Kirim email  
    private void sendInvoice(Order order) {  
        emailService.sendInvoiceEmail(order.getCustomer().getEmail(), order);  
    }  
    // 6. Generate invoice  
    private void generateInvoice(Order order) {  
        pdfService.generateInvoice(order);  
    }  
}
```

KEUNTUNGAN:

- ✓ Pendek (setiap function 5-10 baris)
- ✓ Satu function = satu tanggung jawab
- ✓ Mudah dipahami
- ✓ Mudah diuji (bisa diuji per function)
- ✓ Mudah diperbaiki
- ✓ Mudah digunakan ulang

Validasi data

Hitung total

Hitung diskon

Simpan ke database

Kirim email

Buat invoice PDF



Setiap function kecil, mudah dipahami dan aman untuk diubah.

KESIMPULAN BAIK



Function kecil membuat kode lebih bersih, mudah dipahami, mudah diuji, dan mudah dipelihara dalam jangka panjang.





FOKUS PADA SATU TUGAS

Satu function = Satu Tujuan = Satu Alasan untuk Berubah

APA MAKSUDNYA?



Function yang baik hanya melakukan satu pekerjaan saja.

- Satu function = satu tujuan
- Tidak mencampur banyak proses berbeda
- Tidak memiliki banyak alasan untuk berubah

MENGAPA HARUS FOKUS PADA SATU TUGAS?

- ✓ Mudah dipahami
- ✓ Mudah diuji
- ✓ Mudah diperbaiki
- ✓ Mudah digunakan ulang
- ✓ Mengurangi bug
- ✓ Kode lebih bersih dan stabil

ANALOGI SEDERHANA



Satu orang melakukan banyak pekerjaan akan kacau dan tidak efisien. Begitu juga function.

KEUNTUNGAN FUNCTION FOKUS



Mudah Dibaca
Alurnya jelas seperti membaca cerita.



Mudah Diuji
Bisa diuji per bagian, tidak harus semuanya.



Mudah Maintenance
Perubahan di satu bagian tidak mempengaruhi bagian lain.



Mudah Digunakan Ulang
Function kecil bisa dipakai di banyak tempat.



Mengurangi Bug
Semakin kecil dan fokus, semakin kecil kemungkinan bug.

✗ CONTOH FUNCTION BURUK (TIDAK FOKUS)

Melakukan banyak tugas sekaligus

- 1 Validasi
- 2 Hitung total
- 3 Simpan database
- 4 Kirim email
- 5 Generate PDF

```

public void processOrder(Order order){
    // validasi
    if(order == null){
        throw new IllegalArgumentException();
    }

    // hitung total
    double total = 0;
    for(OrderItem item : order.getItems()){
        total += item.getPrice() * item.getQuantity();
    }

    // simpan database
    orderRepository.save(order);

    // kirim email
    emailService.sendInvoice(order);

    // generate PDF
    pdfService.generateInvoice(order);
}

```

MASALAH:

- ✗ Terlalu panjang
- ✗ Banyak tanggung jawab
- ✗ Sulit diuji
- ✗ Sulit diperbaiki
- ✗ Perubahan kecil bisa mempengaruhi bagian lain
- ✗ Sulit digunakan ulang



Banyak alasan berubah = kode rapuh dan sulit dirawat

✓ CONTOH FUNCTION BAIK (FOKUS PADA SATU TUGAS)

Setiap function hanya melakukan satu tugas

```

public void processOrder(Order order){
    validateOrder(order);
    double total = calculateTotal(order);
    saveOrder(order);
    sendInvoice(order);
    generateInvoicePdf(order);
}

```

ALUR JELAS:

- 1 Validate
- 2 Calculate
- 3 Save
- 4 Send Email
- 5 Generate PDF

```

1 validateOrder()
private void
validateOrder(Order
order){
    if(order == null){
        throw new
IllegalArgumentException
Exception();
    }
}

```

Hanya validasi data

```

2 calculateTotal()
private double
calculateTotal(Order
order){
    double total = 0;
    for(OrderItem item :
order.getItems()){
        total += item.getPrice()
* item.getQuantity();
    }
    return total;
}

```

Hanya menghitung total

```

3 saveOrder()
private void
saveOrder(Order order){
    orderRepository
.save(order);
}

```

Hanya menyimpan data

```

4 sendInvoice()
private void
sendInvoice(Order
order){
    emailService
.sendInvoice(order);
}

```

Hanya mengirim email

```

5 generateInvoicePdf()
private void
generateInvoicePdf(Order
order){
    pdfService
.generateInvoice(order);
}

```

Hanya generate PDF

TANDA FUNCTION TERLALU BESAR

- ⚠ Panjang dan sulit di-scroll
- ⚠ Banyak if/else dan loop
- ⚠ Banyak komentar section (// ---)
- ⚠ Nama function sulit dibuat
- ⚠ Sulit menjelaskan apa yang dilakukan



TEKNIK MEMPERBAIKI

Extract Method

Pisahkan bagian kode menjadi function baru yang lebih kecil dan fokus.

SEBELUM (BURUK)

```

void checkout(){
    validate();
    calculate();
    save();
    sendEmail();
}

```

SESUDAH (BAIK)

```

void validateCart() { }
void calculatePayment() { }
void saveTransaction() { }
void sendInvoiceEmail() { }

```

HUBUNGAN DENGAN SRP

Single Responsibility Principle (SRP)
"One reason to change."

Jika function melakukan banyak hal, maka ada banyak alasan berubah → melanggar SRP.



PESAN UNCLE BOB

“Functions should do one thing. They should do it well. They should do it only.”

— Robert C. Martin



KESIMPULAN

Function yang fokus pada satu tugas membuat kode lebih mudah dipahami, diuji, diperbaiki, digunakan ulang, dan mengurangi bug. Semakin kecil dan fokus sebuah function, semakin mudah software dipelihara dalam jangka panjang.



CHAPTER 3

NAMA HARUS DESKRIPTIF

Nama function harus menjelaskan “APA” yang dilakukan, bukan “BAGAIMANA” caranya.



Pilih nama yang menjelaskan tujuan. Nama yang baik menghilangkan ambiguitas dan membutuhkan sedikit komentar.

– Uncle Bob (Robert C. Martin)

APA MAKSUDNYA?



Nama harus deskriptif = Nama function menjelaskan tujuan/tanggung jawab function tersebut.

- Memberi tahu pembaca “apa” yang dilakukan function
- Menghilangkan kebingungan
- Menghemat waktu membaca dan memahami kode

MENGAPA PENTING?

- Kode lebih mudah dipahami
- Mengurangi pertanyaan dan diskusi
- Mengurangi komentar yang tidak perlu
- Mempercepat onboarding anggota baru
- Mengurangi bug karena salah paham

NAMA YANG BAIK VS NAMA YANG BURUK

NAMA BURUK (AMBIGU)

- processData()
- handle()
- update()
- execute()
- calc()
- doStuff()



NAMA BAIK (DESKRIPTIF)

- processCustomerData()
- handleLoginRequest()
- updateCustomerProfile()
- executeMonthlyReport()
- calculateTotalPrice()
- sendOrderConfirmation()

AMBIGUOUS LANGUAGE

Ambiguous = memiliki banyak makna atau interpretasi berbeda.

Contoh ambiguitas:

- Data apa?
- Kapan di-update?
- Handle apa?
- Execute apa?



Ambiguitas menyebabkan: salah implementasi, bug, dan komunikasi yang buruk.

✗ CONTOH KODE: NAMA AMBIGU (TIDAK DESKRIPTIF)

Contoh: Sistem E-Commerce

```
1 class OrderService {
2   public void process(Order order) {
3     // validasi
4     if (order == null) {
5       throw new IllegalArgumentException();
6     }
7     // hitung total
8     double result = calc(order);
9
10    // simpan
11    save(order);
12
13    // kirim notifikasi
14    send(order);
15  }
16
17  private double calc(Order order) { ... }
18  private void save(Order order) { ... }
19  private void send(Order order) { ... }
20 }
21
```

MASALAH:

- Tidak jelas apa yang diproses
- calc menghitung apa?
- save menyimpan ke mana?
- send mengirim apa?
- Sulit dipahami tanpa membaca isi function
- Membutuhkan banyak komentar



process itu proses apa?
calc itu menghitung apa?
save ke mana?
send apa?

✓ CONTOH KODE: NAMA DESKRIPTIF (JELAS TUJUAN)

Contoh: Sistem E-Commerce

```
1 class OrderService {
2   public void placeOrder(Order order) {
3     validateOrder(order);
4     double totalAmount = calculateOrderTotal(order);
5     saveOrderToDatabase(order, totalAmount);
6     sendOrderConfirmationEmail(order);
7     createInvoiceForOrder(order);
8   }
9
10  private void validateOrder(Order order) { ... }
11  // Hanya validasi data order
12
13  private double calculateOrderTotal(Order order) { ... }
14  // Hanya menghitung total order
15
16  private void saveOrderToDatabase(Order order, double totalAmount) { ... }
17  // Hanya menyimpan order ke database
18
19  private void sendOrderConfirmationEmail(Order order) { ... }
20  // Hanya mengirim email konfirmasi
21
22  private void createInvoiceForOrder(Order order) { ... }
23  // Hanya membuat invoice (PDF/HTML)
24 }

```

KEUNTUNGAN:

- Jelas tujuan setiap function
- Mudah dipahami tanpa membaca seluruh isi function
- Tidak perlu banyak komentar
- Mudah diuji (unit test per function)
- Mudah dipelihara dan diubah
- Mengurangi risiko salah paham



Wah, jelas sekali!
placeOrder, validateOrder,
calculateOrderTotal, dll.
Mudah dipahami!

PRINSIP PENAMAAN BAIK



- Gunakan kata kerja di awal (verb)
- Jelaskan apa yang dilakukan, bukan bagaimana
- Gunakan kata yang mudah dicari dan diucapkan
- Hindari singkatan yang tidak umum
- Satu konsep = satu kata

CONTOH TRANSFORMASI NAMA

NAMA BURUK (AMBIGU)

getData()
update()
delete()
process()
list()

→
→
→
→
→

NAMA BAIK (DESKRIPTIF)

getCustomerData()
updateProductStock()
deleteUserAccount()
processPayment()
getActiveUserList()

DAMPAK NAMA YANG BURUK

- Membuat kode sulit dipahami
- Meningkatkan beban kognitif
- Memperlama waktu debugging
- Menyebabkan salah implementasi
- Menghambat kolaborasi tim

KESIMPULAN



Nama yang deskriptif adalah bentuk komunikasi terbaik dalam kode.

Kode yang baik = Komunikasi yang baik
Komunikasi yang baik = Tim yang produktif

Ingat: Kode adalah untuk manusia yang akan memeliharanya di masa depan, bukan untuk komputer.



Gunakan jumlah parameter sesedikit mungkin. Setiap parameter menambah kompleksitas.

“ Functions should have few arguments.
– Uncle Bob (Robert C. Martin)

APA MAKSUDNYA?



Parameter minimal berarti:

- ✓ Gunakan hanya parameter yang benar-benar dibutuhkan
- ✓ Semakin sedikit parameter, semakin mudah dipahami
- ✓ Mengurangi risiko salah kirim data
- ✓ Memudahkan pemanggilan function
- ✓ Memudahkan pengujian (unit test)

MENGAPA PENTING?



- ✓ Terlalu banyak parameter = sulit dibaca
- ✓ Sulit diingat urutannya
- ✓ Mudah tertukar saat memanggil function
- ✓ Meningkatkan kemungkinan error
- ✓ Sulit melakukan perubahan di masa depan

PRINSIP DASAR



Gunakan parameter hanya untuk:

- 1 Data yang diperlukan untuk tugas function tersebut
- 2 Bukan untuk konfigurasi global
- 3 Bukan untuk data yang bisa diambil dari object lain
- 4 Bukan untuk flag/boolean yang ambigu

CONTOH BURUK: TERLALU BANYAK PARAMETER

Contoh: Generate Laporan Penjualan

```
1 void generateSalesReport(
2     String startDate,
3     String endDate,
4     String region,
5     String productCategory,
6     boolean includeTax,
7     boolean includeDiscount,
8     String outputFormat,
9     String filePath
10 ) {
11     // ... banyak logika
12 }
```

MASALAH:

- ✗ Sulit dibaca dan dipahami
- ✗ Sulit diingat urutannya
- ✗ Mudah tertukar saat dipanggil
- ✗ Jika ada perubahan, banyak tempat yang harus diubah
- ✗ Testing menjadi lebih rumit



Parameter apa saja yang wajib diisi?
Urutannya bagaimana?
Sulit sekali!

TIPS MENGURANGI PARAMETER

- 1 Gabungkan parameter yang berhubungan menjadi satu object (DTO)
- 2 Ambil data dari class lain (member variable / service)
- 3 Hindari parameter boolean (gunakan method terpalah atau enum)
- 4 Gunakan builder pattern jika parameternya opsional
- 5 Pertimbangkan default value jika memungkinkan

CONTOH BAIK: PARAMETER MINIMAL

Solusi 1: Gunakan Object (DTO) untuk mengelompokkan data

```
1 class SalesReportRequest {
2     LocalDate startDate;
3     LocalDate endDate;
4     String region;
5     String productCategory;
6     boolean includeTax;
7     boolean includeDiscount;
8     String outputFormat;
9     String filePath;
10 }
```



```
1 void generateSalesReport(SalesReportRequest request) {
2     // ... banyak logika
3 }
```

KEUNTUNGAN:

- ✓ Hanya 1 parameter
- ✓ Lebih mudah dibaca
- ✓ Urutan data tidak masalah
- ✓ Mudah dikembangkan (tambah field di object saja)
- ✓ Mengurangi risiko salah kirim data

Solusi 2: Ambil data dari object lain (jika tersedia)

```
1 class SalesService {
2     private SalesRepository salesRepository;
3     private UserContext userContext; // punya informasi region, role, dll
4
5     void generateSalesReport(LocalDate start, LocalDate end) {
6         String region = userContext.getRegion();
7         String role = userContext.getRole();
8         // ...
9     }
10 }
```

KEUNTUNGAN:

- ✓ Parameter hanya yang benar-benar dibutuhkan
- ✓ Data lain diambil dari object/service
- ✓ Lebih sederhana dipanggil
- ✓ Lebih mudah dipelihara

CONTOH POLA PARAMETER BURUK LAINNYA

1 Terlalu Banyak Parameter Primitive

```
void createUser(String name, String email,
String phone, String address, String city,
String country, String zipCode, boolean active)
{ ... }
```

- ✗ Susah dibaca, sulit diingat

2 Menggunakan Parameter Boolean

```
void processOrder(Order order, boolean isUrgent) { ... }
```

- ✗ Masalah:
isUrgent = true artinya apa?
Buat method terpalah lebih jelas

- ✓ Solusi:
void processNormalOrder(Order order)
void processUrgentOrder(Order order)

3 Parameter yang Sebenarnya Tidak Perlu

```
void sendEmail(String to, String subject,
String message, EmailService service) { ... }
```

- ✗ Masalah:
EmailService seharusnya member variable,
bukan parameter.

- ✓ Solusi:
void sendEmail(String to, String subject, String message)

4 Parameter Konfigurasi Global

```
void connect(String host, int port,
String username, String password,
int timeout, boolean useSSL) { ... }
```

- ✗ Masalah:
Konfigurasi global tidak perlu dikirim berulang.

- ✗ Solusi:
✓ Gunakan configuration object / application.yml

DAMPAK PARAMETER BERLEBIHAN

- ✗ Kode sulit dipahami
- ✗ Mudah terjadi kesalahan
- ✗ Perubahan kecil berdampak besar
- ✗ Testing lebih sulit
- ✗ Menurunkan kualitas kode

KESIMPULAN



Gunakan jumlah parameter sesedikit mungkin.
Parameter yang sedikit membuat kode lebih
jelas, aman, dan mudah dipelihara.

ⓘ Ingat: Parameter adalah kontrak function. Semakin sedikit kontraknya, semakin mudah digunakan dan semakin kecil kemungkinan membuat kesalahan.



Function hanya melakukan tugas utamanya saja, tanpa diam-diam mengubah hal lain di luar yang diharapkan.

“Functions should do one thing.
They should do it well.
They should do it only.”
— Uncle Bob (Robert C. Martin)

APA ITU SIDE EFFECT?

Side effect terjadi ketika sebuah function selain menjalankan tugas utamanya, juga mengubah sesuatu di luar dirinya (tanpa terlihat jelas dari namanya).

- 🗄 Mengubah data global / state
- 📄 Mengubah object lain
- 📁 Menulis file / database
- ✉ Mengirim email / log
- 🔒 Mengubah session, cache, dll

ANALOGI SEDERHANA

Tombol bertuliskan "Lihat Profil" tetapi saat ditekan ternyata:



- ⚠ Menghapus data
- ⚠ Logout akun
- ⚠ Mengubah password

Nama dan perilakunya tidak sesuai ekspektasi. Itu yang disebut **SIDE EFFECT**.

MENGAPA BERBAHAYA?



- ✔ Sulit diprediksi
- ✔ Sulit debugging
- ✔ Sulit testing
- ✔ Bug muncul tiba-tiba
- ✔ Programmer lain bisa salah menggunakan function

✗ CONTOH SIDE EFFECT (BURUK)

1. Mengubah Global Variable

```
int total = 0;

void addPrice(int price){
    total += price; // mengubah global variable
}
```

MASALAH:

- addPrice() diam-diam mengubah variable global total
- Sulit diprediksi
- Sulit debugging & testing

2. Nama Menipu (Mengubah Session)

```
boolean checkPassword(String username, String password){
    Session.currentUser = username; // side effect
    return true;
}
```

MASALAH:

- Nama checkPassword() terlihat hanya mengecek password
- Padahal mengubah session login diam-diam

3. Terlalu Banyak Side Effects

```
void processOrder(Order order){
    saveToDatabase(order);
    emailService.send(order);
    logService.write(order);
    cache.clear();
    globalCounter++;
}
```

MASALAH:

- Mengubah banyak hal sekaligus: database, email, log, cache, global variable
- Sulit testing (harus mock banyak)
- Sulit debugging
- Sulit dipahami

✔ VERSI TANPA SIDE EFFECT (BAIK)

1. Tanpa Global Variable (Pure Function)

```
int addPrice(int currentTotal, int price){
    return currentTotal + price; // tidak mengubah
                                // data di luar function
}
```

KEUNTUNGAN:

- ✔ Tidak mengubah state luar (pure function)
- ✔ Aman, mudah diuji, mudah dipahami

2. Pisahkan Tugas (Jelas & Terpisah)

```
boolean isPasswordValid(String username, String password){
    // hanya mengecek password
    return true;
}

void loginUser(String username){
    // tugas login dipisahkan
    Session.currentUser = username;
}
```

Hanya tugas validasi

Tugas login terpisah

3. Pecah Menjadi Function Kecil (Satu Tugas)

```
void processOrder(Order order){
    validateOrder(order);
    saveOrderToDatabase(order);
    sendOrderEmail(order);
    writeOrderLog(order);
    clearCache();
    increaseOrderCounter();
}
```

KEUNTUNGAN:

- ✔ Setiap function satu tugas
- ✔ Mudah dipahami
- ✔ Mudah diuji
- ✔ Mudah dipelihara
- ✔ Lebih aman dan stabil

TANDA-TANDA SIDE EFFECT

- ✗ Mengubah global variable
- ✗ Mengubah object eksternal
- ✗ Menulis ke database / file
- ✗ Mengirim email / log
- ✗ Mengubah session / login
- ✗ Mengubah cache / konfigurasi

Tanpa terlihat jelas dari namanya

PURE FUNCTION (IDEAL)

Input sama → Output sama
Tidak mengubah state luar

```
int multiply(int a, int b){
    return a * b;
}
```

Keuntungan:

- ✔ Aman
- ✔ Thread-safe
- ✔ Mudah dites
- ✔ Dapat digunakan ulang
- ✔ Mudah dipahami

CARA HINDARI SIDE EFFECT

1. Buat function sekecil mungkin dan fokus pada satu tugas.
2. Jangan mengubah state di luar kecuali memang itu tugasnya.
3. Pisahkan function yang punya tanggung jawab berbeda.
4. Gunakan pure function jika memungkinkan.
5. Beri nama function yang menjelaskan apa yang dilakukan, bukan bagaimana.

DAMPAK POSITIF

- 📖 Kode lebih mudah dibaca
- 📋 Lebih mudah diuji (unit test)
- 🕒 Debugging lebih cepat
- 🛡 Risiko bug lebih kecil
- 🔒 Kode lebih aman dan stabil
- 👥 Mudah dikerjakan oleh tim



KESIMPULAN

Hindari side effects berarti membuat function fokus pada tugasnya, tanpa efek samping tersembunyi. Hasilnya adalah kode yang bersih, aman, mudah dipahami, dan mudah dipelihara dalam jangka panjang.



Chapter 4 - Comments





Uncle Bob
(Robert C. Martin)

Komentar bukan solusi untuk kode buruk.
Kode yang baik seharusnya bisa “berbicara sendiri”.

“

Comments Do Not Make Up for Bad Code

Komentar tidak bisa memperbaiki kode yang buruk.

MENGAPA UNCLE BOB TIDAK SUKA BANYAK KOMENTAR?

Karena komentar sering:

- | | |
|-------------------|--|
| ✗ Usang | Komentar tidak di-update saat kode berubah |
| ✗ Bohong | Komentar bisa tidak sesuai dengan kode |
| ✗ Tidak di-update | Sering terlambat saat melakukan perubahan |
| ✗ Menyesatkan | Membuat programmer salah paham |
| ✗ Redundant | Mengulang hal yang sudah jelas dari kode |

KODE YANG BAIK

Seharusnya bisa dibaca dan dipahami tanpa harus bergantung pada komentar.



Kode yang jelas = Komentar yang sedikit

CONTOH BURUK: KOMENTAR MENUTUPI KODE BURUK

```
// menghitung total harga
double t = q * p;
```

MASALAH:

- ✗ Variabel **t**, **q**, **p** tidak jelas
- ✗ Komentar dipakai untuk menutupi naming buruk

VERSI CLEAN CODE: KODE BICARA SENDIRI

```
double totalPrice = quantity * price;
```



Tanpa komentar pun:
langsung dipahami.

CONTOH KOMENTAR BOHONG (BERBAHAYA)

```
// Mengurutkan data ascending
sortDescending(data);
```



Komentar tidak sesuai kode.
Ini berbahaya karena bisa
menyesatkan programmer lain.

LEBIH BAIK JELASKAN DENGAN KODE

✗ BURUK (Butuh Komentar)

```
// cek apakah user aktif
if(user.status == 1)
```



✓ BAIK (Tanpa Komentar)

```
if(user.isActive())
```

ALASAN:

- ✓ Nama function jelas
- ✓ Mudah dipahami
- ✓ Tidak perlu komentar

JENIS KOMENTAR YANG BURUK (SEBAIKNYA DIHINDARI)

1. Redundant Comment

Mengulang kode

```
// increment i
i++;
```

- ✗ Tidak memberi nilai tambah.

2. Misleading Comment

Menyesatkan

```
// mengambil semua user aktif
List<User> users = getAllUsers();
```

- ✗ Komentar tidak sesuai dengan perilaku kode.

3. Commented-Out Code

Kode lama yang di-comment

```
// saveToDatabase(user);
```

- ✗ Hapus saja.
Gunakan version control (Git).

4. Noise Comments

Tidak penting

```
/**
 * Class User
 */
public class User {
```

- ✗ Tidak memberi informasi baru.

5. Journal Comments

Riwayat perubahan

```
// 2024: diperbaiki login
// 2025: ganti query
```

- ✗ Riwayat ada di Git,
bukan di komentar.

JENIS KOMENTAR YANG MASIH BAIK (BOLEH DIGUNAKAN)

1. Legal Comment

Lisensi / copyright

```
/*
 * Copyright 2025
 * All rights reserved
 */
```

- ✓ Diperlukan
secara legal.

2. Warning Comment

Peringatan penting

```
// Jangan hapus cache
// sebelum transaksi
// selesai
```

- ✓ Memberi peringatan
risiko serius.

3. TODO Comment

Pekerjaan belum selesai

```
// TODO: optimasi
// query database
```

- ✓ Menandai pekerjaan
yang belum selesai.

4. Explanation of Intent

Menjelaskan alasan
(kenapa, bukan apa)

```
// Menggunakan timeout 30 detik
// karena API pihak ketiga
// sering lambat
```

- ✓ Menjelaskan alasan
bisnis / logika kompleks.

PRINSIP UTAMA

Komentar seharusnya:

- ✓ Singkat
- ✓ Selalu relevan
- ✓ Jelas
- ✓ Menjelaskan alasan kompleks
- ✓ Penting

KOMENTAR TIDAK SEHARUSNYA

- ✗ Menutupi kode buruk
- ✗ Menggantikan naming yang baik
- ✗ Menjelaskan hal yang obvious
- ✗ Menjadi dokumentasi palsu
- ✗ Tidak relevan / usang

FILOSOFI UNCLE BOB

“The proper use of comments is to compensate for our failure to express ourself in code.

Komentar digunakan ketika kode gagal menjelaskan dirinya sendiri.



KESIMPULAN



Kode yang bersih lebih baik daripada kode buruk yang penuh komentar.

Buat kode yang jelas, sehingga komentar menjadi sedikit namun bermakna.



CHAPTER 4 COMMENTS



Uncle Bob
(Robert C. Martin)

Comments Do Not Make Up for Bad Code.

Komentar bukan solusi utama untuk menjelaskan kode buruk.

→ Kode yang baik seharusnya bisa "berbicara sendiri".

MENGAPA UNCLE BOB TIDAK SUKA BANYAK KOMENTAR?

Karena komentar sering:



Usang



Bohong



Tidak
di-update



Menyesatkan



Redundant
(tidak perlu)

KOMENTAR BOHONG (MENYESATKAN)

```
// Mengurutkan data ascending  
sortDescending(data);
```

✗ Komentar tidak sesuai kode. Ini berbahaya.

Programmer bisa salah memahami perilaku kode.

JENIS KOMENTAR YANG BURUK (HINDARI!!)

1 Redundant Comment

Komentar yang hanya mengulang kode.

✗ Buruk

```
// increment i  
i++;
```

Komentar tidak memberi nilai tambah.



2 Misleading Comment

Komentar menyesatkan.

✗ Buruk

```
// mengambil semua user aktif  
List<User> users = getAllUsers();
```

Padahal:

- mengambil semua user, bukan hanya aktif.



3 Commented-Out Code

Kode lama yang di-comment.

✗ Buruk

```
// saveToDatabase(user);
```

Menurut Uncle Bob:

- hapus saja,
- gunakan version control (Git).



4 Noise Comments

Komentar tidak penting.

✗ Buruk

```
/*  
 * Class User  
 */  
public class User { }
```

Tidak memberi informasi baru.



5 Journal Comments

Komentar riwayat perubahan.

✗ Buruk

```
// 2024: diperbaiki login  
// 2025: ganti query
```

Riwayat seharusnya ada di:

- ✓ Git
- ✓ Version Control



JENIS KOMENTAR YANG MASIH DIANGGAP BAIK (GUNAKAN DENGAN BIJAK)

1 Legal Comment

Komentar lisensi/copyright.

```
/*  
 * Copyright 2025  
 * All rights reserved  
 */
```



2 Warning Comment

Memberi peringatan penting.

```
// Jangan hapus cache  
sebelum transaksi  
selesai
```



Karena ada risiko serius.

3 TODO Comment

Menandai pekerjaan yang belum selesai.

```
// TODO: optimasi  
query database
```



4 Explanation of Intent

Menjelaskan alasan bisnis/logika kompleks.

```
// Menggunakan timeout 30 detik  
// karena API pihak ketiga  
// sering lambat
```



Ini membantu memahami: "kenapa" (alasan) bukan sekadar "apa" (apa yang terjadi).

HUBUNGAN DENGAN NAMING

✗ Buruk (Nama tidak jelas, butuh komentar)

```
// menghitung total pajak  
double x = y * 0.11;
```



Apa itu x, y?
Sulit dipahami!

✓ Lebih Baik (Nama jelas, tidak butuh komentar)

```
double taxAmount = price * TAX_PERCENTAGE;
```



Jelas!
Langsung dipahami.

PRINSIP UTAMA TENTANG COMMENTS

Komentar seharusnya:

- ✓ Singkat
- ✓ Selalu relevan
- ✓ Jelas
- ✓ Menjelaskan alasan kompleks
- ✓ Penting

Komentar TIDAK seharusnya:

- ✗ Menutupi kode buruk
- ✗ Menggantikan naming yang baik
- ✗ Menjelaskan hal yang obvious
- ✗ Menjadi dokumentasi palsu

FILOSOFI UNCLE BOB

“The proper use of comments is to compensate for our failure to express ourself in code.”



Komentar digunakan ketika kode gagal menjelaskan dirinya sendiri.

— Uncle Bob

KESIMPULAN



Kode yang baik harus bisa "berbicara sendiri".
Gunakan komentar hanya ketika benar-benar diperlukan.



Komentar yang baik membuat kode lebih mudah dipahami.
Kode yang baik membuat komentar hampir tidak diperlukan.

Intinya: Tulis kode yang jelas, gunakan nama yang bermakna, dan beri komentar hanya untuk hal yang penting dan tidak bisa dijelaskan dengan kode.



Chapter 5 - Formating



CHAPTER 5

FORMATTING



Uncle Bob
(Robert C. Martin)

Tujuan Formatting: Membuat kode mudah dibaca, dipahami, dan terlihat profesional.

“Code is read much more often than it is written.
So, make it easy to read.”
– Uncle Bob

KONSEP UTAMA

- ✓ **Konsistensi**
Format kode harus konsisten di seluruh proyek.
- ✓ **Keterbacaan**
Format yang baik membantu pembaca memahami struktur dan maksud kode.
- ✓ **Komunikasi**
Format adalah cara kita berkomunikasi dengan tim melalui kode.



ATURAN FORMATTING PENTING

1. Gunakan Indentasi yang Konsisten Gunakan indentasi (biasanya 4 space) untuk menunjukkan struktur kode.	✗ BURUK <pre>if(a>0){ for(int i=0;i<10;i++){ doSomething(); } }</pre>	✓ BAIK <pre>if (a > 0) { for (int i = 0; i < 10; i++) { doSomething(); } }</pre>
2. Batasi Panjang Baris Usahakan tiap baris kode tidak terlalu panjang (maksimal 100-120 karakter).	<pre>String user = getUserService().getReposit ById(id).getProfile().getAddress().getCity();</pre>	<pre>String city = getUserService() .getRepository() .findUserById(id) .getProfile() .getAddress() .getCity();</pre>
3. Gunakan Spasi yang Konsisten Gunakan spasi di sekitar operator, setelah koma, dan setelah kata kunci kontrol.	<pre>if(a>0){ intTotal=a+b; for(int i=0;i<10;i++){ doSomething(); } }</pre>	<pre>if (a > 0) { int total = a + b; for (int i = 0; i < 10; i++) { doSomething(); } }</pre>
4. Letakkan Kurung Kurawal di Baris yang Sama Kurung kurawal pembuka diletakkan di baris yang sama dengan pernyataan kontrol.	<pre>if (a > 0) { doSomething(); }</pre>	<pre>if (a > 0) { doSomething(); }</pre>
5. Beri Spasi Kosong untuk Pemisah Logis Gunakan baris kosong untuk memisahkan blok kode yang berbeda secara logis.	<pre>public void process(){ int a = 0; int b = 1; int c = a + b; print(c); log(c); }</pre>	<pre>public void process() { int a = 0; int b = 1; int c = a + b; print(c); log(c); }</pre>
6. Kelompokkan dengan Logis dan Urutan Vertikal Letakkan variabel yang terkait berdekatan dan urutkan secara vertikal agar mudah dipindai.	<pre>int z = 0; String name = "Bob"; int a = 1; String email = "bob@mail.com"; int b = 2;</pre>	<pre>int a = 1; int b = 2; int z = 0; String name = "Bob"; String email = "bob@mail.com";</pre>

CONTOH PERBANDINGAN KODE SECARA KESELURUHAN

✗ BURUK (Sulit Dibaca)

```
public int hitungTotal(List<Item>items){int
total=0;for(int i=0;i<items.size();i++){if
(items.get(i).isAktif())){total+=items.get(i)
.getHarga()*items.get(i).getQty();}return
total;}
```

- ✗ Semua menempel, sulit dibaca
- ✗ Tidak ada indentasi yang jelas
- ✗ Terlihat melelahkan untuk mata

✓ BAIK (Mudah Dibaca)

```
public int hitungTotal(List<Item> items) {
  int total = 0;

  for (int i = 0; i < items.size(); i++) {
    Item item = items.get(i);
    if (item.isAktif()) {
      total += item.getHarga() * item.getQty();
    }
  }

  return total;
}
```

- ✓ Struktur jelas
- ✓ Mudah dipahami
- ✓ Mata tidak cepat lelah

GUNAKAN TOOLS

Gunakan code formatter / linter otomatis untuk menjaga konsistensi:

- IntelliJ IDEA / Eclipse (Formatter)
- Prettier (JavaScript/TypeScript)
- Black (Python)
- EditorConfig
- ESLint / Checkstyle / Spotless



CHECKLIST FORMATTING

- ✓ Indentasi konsisten
- ✓ Panjang baris tidak berlebihan
- ✓ Spasi digunakan dengan tepat
- ✓ Kurung kurawal diletakkan dengan benar
- ✓ Baris kosong untuk pemisah logis
- ✓ Pengelompokan kode secara vertikal
- ✓ Konsisten di seluruh file proyek

PRINSIP PENTING



- ✓ Format yang baik = Menghargai pembaca kode.
- ✓ Konsistensi lebih penting daripada gaya pribadi.
- ✓ Ikuti standar tim atau gunakan formatter otomatis.

INGAT!



Kita menulis kode untuk komputer, tetapi kita memelihara kode untuk manusia.
Buatlah kode yang enak dibaca!

KESIMPULAN

Formatting bukan sekadar estetika. Kode yang diformat dengan baik akan:

- ✓ Lebih mudah dibaca
- ✓ Lebih mudah dipelihara
- ✓ Mengurangi kesalahan komunikasi dalam tim



Chapter 6 - Objects and Data Structures



CHAPTER 6 OBJECTS AND DATA STRUCTURES



“The difference between objects and data structures is subtle but important.”

— Uncle Bob

INTI PEMBAHASAN



Pilih struktur data sederhana untuk menyimpan data.
Gunakan objek hanya jika ada perilaku (behavior) yang berarti.
Jangan menambahkan metode hanya untuk membungkus data (“anemic objects”).

PRINSIP UTAMA

- ✓ Data adalah data. Simpan dalam struktur data.
- ✓ Perilaku yang bermakna seharusnya berada di objek.
- ✓ Objek bukan sekadar wadah data.
- ✓ Hindari objek tanpa perilaku (Anemic Domain Model).
- ✓ Gunakan data structure ketika tidak ada behavior.



1. DATA STRUCTURE: HANYA MENYIMPAN DATA

Gunakan untuk data passive yang tidak memiliki behavior berarti.

```
class Point {  
    public int x;  
    public int y;  
}
```



CIRI DATA STRUCTURE

- ✓ Tidak memiliki behavior berarti
- ✓ Field public
- ✓ Tidak ada encapsulation
- ✓ Hanya menyimpan data

VS

CONTOH

Menyimpan koordinat titik, jumlah, flag, konfigurasi, record, DTO (Data Transfer Object), dll.

2. OBJECT: MEMILIKI BEHAVIOR YANG BERMAKNA

Gunakan ketika data memiliki behavior (tanggung jawab).

```
class BankAccount {  
    private double balance;  
  
    public void deposit(double amount) {  
        balance += amount;  
    }  
  
    public void withdraw(double amount) {  
        balance -= amount;  
    }  
  
    public double getBalance() {  
        return balance;  
    }  
}
```

CIRI OBJECT

- ✓ Memiliki behavior/metode
- ✓ Encapsulation (field private)
- ✓ Melindungi data
- ✓ Memiliki tanggung jawab yang jelas
- ✓ Perilaku dan data berada dalam satu kesatuan

CONTOH

Akun bank, Order, User, Invoice, Payment, semuanya punya perilaku nyata.

3. PERBANDINGAN CEPAT

	DATA STRUCTURE	OBJECT
Tujuan	Menyimpan data	Mengelola data dan perilaku
Behavior	Tidak ada / sangat sedikit	Ada dan bermakna
Encapsulation	Biasanya tidak	Ya, lindungi data
Contoh	Point, Rectangle, DTO	User, Order, Account
Kapan digunakan	Data pasif tanpa aturan bisnis	Ada aturan bisnis / logic
Risiko jika salah	Over-engineering	Anemic Object (objek lemah)

4. HINDARI ANEMIC DOMAIN MODEL (OBJEK LEMAH)

Objek hanya berisi field dan getter/setter tanpa behavior berarti.

```
class Order {  
    private List<OrderItem> items;  
    private Customer customer;  
    private LocalDate date;  
    // hanya getter/setter...  
}
```

MASALAH

- ✗ Tidak ada behavior
- ✗ Logika tersebar di service
- ✗ Sulit dipahami dan diubah
- ✗ Melanggar prinsip OOP

LEBIH BAIK

```
class Order {  
    private List<OrderItem> items;  
    private Customer customer;  
    private LocalDate date;  
  
    public double calculateTotal() { ... }  
    public void additem(OrderItem item) { ... }  
    public boolean isOverdue() { ... }  
}
```

- ✓ Memiliki behavior yang relevan
- ✓ Logika berada di tempat yang tepat
- ✓ Lebih mudah dirawat dan diuji
- ✓ Domain lebih kaya dan jelas

5. KAPAN MENGGUNAKAN DATA STRUCTURE vs OBJECT?

Apakah data ini memiliki behavior/tanggung jawab yang bermakna?

TIDAK



TOAK

Gunakan DATA STRUCTURE

Contoh:

Point, Size, Config, DTO,

Record, Response, dll.

YA

Gunakan OBJECT

Contoh:

User, Order, Account,

Payment, Product, dll.



Ingat: Behavior yang bermakna = ada aturan bisnis, perubahan state, validasi, perhitungan penting, dll.

6. CONTOH TRANSFORMASI YANG BAIK

DARI DATA STRUCTURE

```
class Rectangle {  
    public int width;  
    public int height;  
}
```

Hanya menyimpan data, tidak ada perilaku.

MENJADI OBJECT

```
class Rectangle {  
    private int width;  
    private int height;  
  
    public int area() {  
        return width * height;  
    }  
  
    public boolean isSquare() {  
        return width == height;  
    }  
}
```

Sekarang memiliki perilaku yang bermakna.

7. TANDA-TANDA ANDA MEMBUAT OBJEK YANG SALAH

- ✗ Objek hanya berisi field dan getter/setter
- ✗ Tidak ada behavior signifikan
- ✗ Semua logika berada di service/utility
- ✗ Jika hapus objek, tidak ada yang berubah



8. MANFAAT MODEL YANG BAIK

- ✓ Kode lebih mudah dipahami
- ✓ Perubahan lebih aman
- ✓ Perilaku dan data terstruktur
- ✓ Domain lebih dekat dengan dunia nyata



9. PESAN UNCLE BOB

“Objects should have behavior. Data structures should have public fields and no behavior.”

— Uncle Bob



10. RINGKASAN

Data pasif tanpa behavior

DATA DATA STRUCTURE



Ada behavior dan aturan bisnis

Gunakan OBJECT



Chapter 7 – Error Handling



Contoh Error Handling try-catch yang Benar

Contoh lain: Membaca file konfigurasi JSON

“Tangani error sekali, dengan konteks yang cukup, dan lakukan sesuatu yang bermakna.” ”

– Robert C. Martin



CONTOH SALAH

Menangkap Exception umum, tanpa konteks, dan tidak ada tindakan berarti.

```
public Config loadConfig(String path) {  
    Config config = null;  
    try {  
        FileReader reader = new FileReader(path);  
        Gson gson = new Gson();  
        config = gson.fromJson(reader, Config.class);  
        reader.close();  
    } catch (Exception e) {  
        // abaikan saja  
    }  
    return config; // bisa jadi null  
}
```

Exception terlalu umum
(bisa menutupi masalah)

Menelan error
(tidak ada log / info)

Return null
(memaksa caller cek null)



CONTOH BENAR

Menangkap exception spesifik, memberi konteks, dan mengambil tindakan.

```
public Config loadConfig(String path) {  
    Objects.requireNonNull(path, "path is required");  
  
    try (Reader reader = new FileReader(path)) {  
        Gson gson = new Gson();  
        return gson.fromJson(reader, Config.class);  
    } catch (FileNotFoundException e) {  
        String msg = "Config file not found: " + path;  
        logger.error(msg, e);  
        throw new ConfigurationException(msg, e);  
    } catch (JsonSyntaxException e) {  
        String msg = "Invalid JSON in config file: " + path;  
        logger.error(msg, e);  
        throw new ConfigurationException(msg, e);  
    } catch (IOException e) {  
        String msg = "Failed to read config file: " + path;  
        logger.error(msg, e);  
        throw new ConfigurationException(msg, e);  
    }  
}
```

Gunakan try-with-resources
agar resource auto-close

Tangkap exception spesifik
sesuai kemungkinan masalah

Berikan konteks (pesan + path)
agar mudah dipahami

Jangan return null,
lempar exception yang
bermakna



PENJELASAN

- Contoh salah menelan error sehingga masalah tersembunyi dan muncul jauh di kemudian hari.
- Contoh benar menangani error di tempat yang tepat, memberi informasi cukup, dan mencegah masalah menyebar.
- Exception spesifik membantu kita memahami jenis masalah.
- Log + pesan yang jelas memprecepat diagnosis.
- Lempar exception khusus (ConfigurationException) agar caller tahu bahwa operasi ini gagal.



PRINSIP CLEAN CODE (ERROR HANDLING)

- Tangani di satu tempat.
- Berikan konteks yang cukup.
- Pisahkan alur normal dari alur error.
- Jangan kembalikan null.
- Gunakan exception untuk hal yang benar-benar exceptional.
- Buat kode yang mudah dibaca dan dirawat.

